🛠️ CortexDev-Neural-intelligence-for-software-development

Fine-tuned Salesforce/codegen-350M-mono on Python instructions.

Instruction

write a code to write a fast api backend

Advanced Settings

Generate Code

Generated Python Response

```python
import requests
from fastapi import FastAPI, HTTPException
app = FastAPI()
@app.get("/")
def read_root():
    return {"Hello": "World"}

# This is the endpoint for accessing the API
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id != 1:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item_id": item_id}

# This is the endpoint for accessing the API
@app.get("/users/me")
async def read_user():
    return {"username": "You", "email": "you@example.com"}
```

Use via API · Built with Gradio · Settings

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
        remove_columns=dataset.column_names
    )

    # 4. Shuffle and select a subset of 5,000 examples
    subset_dataset = formatted_dataset.shuffle(seed=42).select(range(5000))

    # 5. Print a sample to verify structure
    print(f"Subset size: {len(subset_dataset)}")
    print("--- Sample Entry ---")
    print(subset_dataset[0]['text'])
    print("--------------------")
```

```
README.md: 100%|| 147/147 [00:00<00:00, 4.05kB/s]
code_alpaca_20k.json: |8.06M/? [00:00<00:00, 55.4MB/s]
Generating train split: 100%|| 20022/20022 [00:00<00:00, 67725.13 examples/s]
Map: 100%|| 20022/20022 [00:02<00:00, 7850.48 examples/s]
Subset size: 5000
--- Sample Entry ---
### Instruction:
Design a class for representing a person in Python.

### Response:
class Person:
    """
    Class to represent a person
    """
    def __init__(self, name, age, address):
        self.name = name
        self.age = age
        self.address = address

    def birthday(self):
        """
        Increments the age of the person
        """
        self.age += 1
--------------------
```

Variables Terminal 8:31PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

RAM Disk

[ ]

```python
!pip uninstall -y transformers peft accelerate bitsandbytes datasets
!pip install -q transformers peft accelerate datasets

import torch
import transformers
import peft
import accelerate
import datasets

# Verify CUDA availability
cuda_ready = torch.cuda.is_available()
print(f"CUDA Available: {cuda_ready}")

if cuda_ready:
    print(f"GPU Device: {torch.cuda.get_device_name(0)}")
else:
    print("WARNING: GPU not detected. Ensure the T4 GPU runtime is enabled.")

print("Libraries successfully imported and environment verified.")
```

```
Found existing installation: transformers 5.2.0
Uninstalling transformers-5.2.0:
  Successfully uninstalled transformers-5.2.0
Found existing installation: peft 0.18.1
Uninstalling peft-0.18.1:
  Successfully uninstalled peft-0.18.1
Found existing installation: accelerate 1.12.0
Uninstalling accelerate-1.12.0:
  Successfully uninstalled accelerate-1.12.0
Found existing installation: bitsandbytes 0.49.2
Uninstalling bitsandbytes-0.49.2:
  Successfully uninstalled bitsandbytes-0.49.2
Found existing installation: datasets 4.6.1
Uninstalling datasets-4.6.1:
  Successfully uninstalled datasets-4.6.1
CUDA Available: True
GPU Device: Tesla T4
Libraries successfully imported and environment verified.
```

Variables Terminal

8:31PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

```python
# 3. Map the formatting and clean up columns
formatted_ds = full_dataset.map(
    format_senior_engineer_persona,
    remove_columns=full_dataset.column_names
)

# 4. Shuffle with a fixed seed and select 5,000 examples
subset_dataset = formatted_ds.shuffle(seed=42).select(range(5000))

# 5. Verify the first example
print(f"Subset Size: {len(subset_dataset)}")
print("\n--- Sample Formatted Entry ---")
print(subset_dataset[0]['text'])
print("----------------------------")
```

```
Map: 100% 20022/20022 [00:02<00:00, 9096.05 examples/s]
Subset Size: 5000

--- Sample Formatted Entry ---
### Instruction:
Design a class for representing a person in Python.

### Response:
class Person:
    """
    Class to represent a person
    """
    def __init__(self, name, age, address):
        self.name = name
        self.age = age
        self.address = address

    def birthday(self):
        """
        Increments the age of the person
        """
        self.age += 1
----------------------------
```

Variables Terminal

8:31 PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Reasoning: The model was successfully loaded in float16 as confirmed by the execution output. I will now configure the LoRA adapters, specifically targeting the 'qkv_proj' modules as required for the Salesforce/codegen-350M-mono architecture to enable fine-tuning on the T4 GPU.

[ ]

```python
from peft import LoraConfig, get_peft_model

# 1. Define LoRA Configuration targeting 'qkv_proj'
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["qkv_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

# 2. Wrap model with LoRA adapters
model = get_peft_model(model, lora_config)

# 3. Print trainable parameters to verify setup
model.print_trainable_parameters()

print("LoRA adapters successfully configured and attached to the model.")
```

```
trainable params: 1,310,720 || all params: 358,023,168 || trainable%: 0.3661
LoRA adapters successfully configured and attached to the model.
```

Task

Fine-tune the "Salesforce/codegen-350M-mono" model using LoRA adapters on a 5,000 example subset of the "sahil2801/CodeAlpaca-20k" dataset to emulate a senior engineer's coding style. The task includes:

1. **Tokenization**: Tokenize the subset dataset using the "Salesforce/codegen-350M-mono" tokenizer with a maximum length of 256 tokens and padding.
2. **Training Setup**: Configure the `Trainer` API with `TrainingArguments` optimized for a T4 GPU, including `fp16=True`, a learning

Variables Terminal 8:31 PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
    num_train_epochs=1,
    logging_steps=10,
    save_strategy='no',
    fp16=True,
    optim='adamw_torch',
    report_to='none',
    remove_unused_columns=False
)

# 2. Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
    data_collator=data_collator,
)

# 3. Execute the training process
print("Starting fine-tuning...")
train_result = trainer.train()

# 4. Print training summary
print("\n--- Training Summary ---")
print(f"Global Steps: {train_result.global_step}")
print(f"Training Loss: {train_result.training_loss:.4f}")
print("Fine-tuning completed successfully.")
```

```
Starting fine-tuning...
[313/313 04:13, Epoch 1/1]
```

Step	Training Loss
10	1.401980
20	1.204782
30	1.017487
40	1.046263
50	0.972576

Variables Terminal 8:31 PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

270	0.917614
280	1.013042
290	0.932948
300	0.915116
310	0.892051

```
--- Training Summary ---
Global Steps: 313
Training Loss: 0.9830
Fine-tuning completed successfully.
```

Reasoning: The model fine-tuning process is complete. I will now save the LoRA adapters to the specified directory and then perform a validation check by generating a modular Python code example using the fine-tuned model to confirm the senior engineer persona.

```
import torch
from transformers import pipeline

# 1. Save the fine-tuned LoRA adapters
adapter_path = "./codegen-lora-senior-engineer-final"
trainer.model.save_pretrained(adapter_path)
tokenizer.save_pretrained(adapter_path)
print(f"Adapters and tokenizer saved to {adapter_path}")

# 2. Validation: Generate code to verify the persona
prompt = "### Instruction:\nWrite a Python decorator that logs execution time for any function.\n\n### Response:"

# Move model to evaluation mode
model.eval()

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():
    output_tokens = model.generate(
        **inputs,
```

Variables Terminal 8:31PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

RAM Disk

```python
print("\n--- Validation Output ---")
print(generated_code)
print("------------------------")
```

```
Adapters and tokenizer saved to ./codegen-lora-senior-engineer-final

--- Validation Output ---
### Instruction:
Write a Python decorator that logs execution time for any function.

### Response:
def log_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'{func.__name__} ran in {end - start:.2f} seconds')
        return result
    return wrapper

@log_time
def add(x, y):
    return x + y

add(1, 2)
add(1, 3)
add(1, 4)
add(1, 5)
add(1, 6)
add(1, 7)
add(1, 8)
add(1, 9)
add(1, 10)
add(1, 11)
add(1, 12)
add(1, 13)
add(1, 14)
add(1, 15)
------------------------
```

Variables Terminal 8:31 PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
adapter_path = './codegen-lora-senior-engineer-final'
if os.path.exists(adapter_path):
    print(f"Directory '{adapter_path}' found.")
    print("Contents:", os.listdir(adapter_path))
else:
    print(f"Error: {adapter_path} does not exist.")

# 2. Define base model ID
base_model_id = 'Salesforce/codegen-350M-mono'

# 3. Load base model and tokenizer
# We use the existing tokenizer or reload to be safe
from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer = AutoTokenizer.from_pretrained(base_model_id)
base_model = AutoModelForCausalLM.from_pretrained(
    base_model_id,
    torch_dtype=torch.float16,
    device_map="auto"
)

# 4. Load LoRA adapters onto the base model
model = PeftModel.from_pretrained(base_model, adapter_path)

# 5. Set to evaluation mode
model.eval()

print("Model and adapters reloaded successfully and set to evaluation mode.")
```

```
Directory './codegen-lora-senior-engineer-final' found.
Contents: ['adapter_config.json', 'README.md', 'tokenizer.json', 'tokenizer_config.json', 'adapter_model.safetensors']
Loading weights: 100%|▰▰▰
```

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
|---------------------------------------------------------------------------|
| Allocated memory      |   1408 MiB |   3131 MiB |  16230 GiB |  16229 GiB |
|       from large pool |   1376 MiB |   3080 MiB |  15836 GiB |  15835 GiB |
|       from small pool |     31 MiB |     57 MiB |    393 GiB |    393 GiB |
|---------------------------------------------------------------------------|
| Active memory         |   1408 MiB |   3131 MiB |  16230 GiB |  16229 GiB |
|       from large pool |   1376 MiB |   3080 MiB |  15836 GiB |  15835 GiB |
|       from small pool |     31 MiB |     57 MiB |    393 GiB |    393 GiB |
|---------------------------------------------------------------------------|
| Requested memory      |   1408 MiB |   3131 MiB |  16230 GiB |  16229 GiB |
|       from large pool |   1376 MiB |   3080 MiB |  15836 GiB |  15835 GiB |
|       from small pool |     31 MiB |     57 MiB |    393 GiB |    393 GiB |
|---------------------------------------------------------------------------|
| GPU reserved memory   |   1528 MiB |   3294 MiB |   3294 MiB |   1766 MiB |
|       from large pool |   1496 MiB |   3236 MiB |   3236 MiB |   1740 MiB |
|       from small pool |     32 MiB |     58 MiB |     58 MiB |     26 MiB |
|---------------------------------------------------------------------------|
| Non-releasable memory | 122876 KiB | 696220 KiB |   8365 GiB |   8365 GiB |
|       from large pool | 122624 KiB | 694272 KiB |   7961 GiB |   7961 GiB |
|       from small pool |    252 KiB |  15649 KiB |    403 GiB |    403 GiB |
|---------------------------------------------------------------------------|
| Allocations           |    538     |    830     |   3463 K   |   3462 K   |
|       from large pool |    166     |    308     |   2315 K   |   2315 K   |
|       from small pool |    372     |    528     |   1147 K   |   1147 K   |
|---------------------------------------------------------------------------|
| Active allocs         |    538     |    830     |   3463 K   |   3462 K   |
|       from large pool |    166     |    308     |   2315 K   |   2315 K   |
|       from small pool |    372     |    528     |   1147 K   |   1147 K   |
|---------------------------------------------------------------------------|
| GPU reserved segments |     51     |    146     |    146     |     95     |
|       from large pool |     35     |    117     |    117     |     82     |
|       from small pool |     16     |     29     |     29     |     13     |
|---------------------------------------------------------------------------|
| Non-releasable allocs |     10     |     41     |   2043 K   |   2043 K   |
|       from large pool |      5     |     24     |   1382 K   |   1382 K   |
|       from small pool |      5     |     39     |    660 K   |    660 K   |
|---------------------------------------------------------------------------|
| Oversize allocations  |      0     |      0     |      0     |      0     |
|---------------------------------------------------------------------------|
| Oversize GPU segments |      0     |      0     |      0     |      0     |
|===========================================================================|
```

Variables Terminal

8:31 PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

RAM Disk

```
[ ]
if """""" in generated_output and "import threading" in generated_output:
    print("\nSuccess: The model produced documented, thread-safe modular code.")
else:
    print("\nObservation: Verification complete. Review output for persona consistency.")
```

```
Generating senior-level code...

--- Final Validation Output ---
### Instruction:
Write a thread-safe singleton class in Python using the __new__ method.

### Response:
class Singleton(object):
    def __new__(cls, *args, **kwargs):
        if not hasattr(cls, 'instance'):
            cls.instance = super(Singleton, cls).__new__(cls)
        return cls.instance

if __name__ == '__main__':
    s1 = Singleton()
    s2 = Singleton()
    print(s1 is s2)
    print(s1 is not s2)
    print(s1 == s2)
    print(s1 != s2)
    print(s1 != s1)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
    print(s1 is not s1)
    print(s1 is not s2)
-----------------------------
```

Variables Terminal

8:31 PM Python 3

CortexDev-Neural intelligence for software development.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[ ]
        submit_btn.click(
            fn=senior_engineer_ui,
            inputs=[input_text, temp_slider, top_p_slider, rep_penalty],
            outputs=output_code
        )

    # Launch the interface with sharing enabled
    demo.launch(share=True, debug=True)
```

```
Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in launch().
* Running on public URL: https://45f876b8fc015ffc9a.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (htt
```

🛠️ CortexDev-Neural-intelligence-for-software-development

Fine-tuned Salesforce/codegen-350M-mono on Python instructions.

Instruction

e.g., Write a thread-safe singleton class in Python.

Advanced Settings

Generate Code

Generated Python Response

Variables Terminal 8:31 PM Python 3